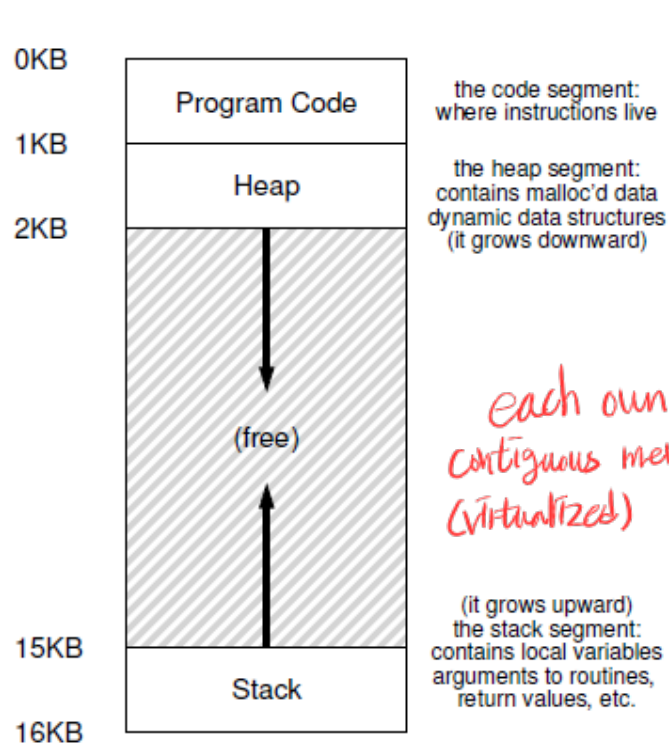

Memory

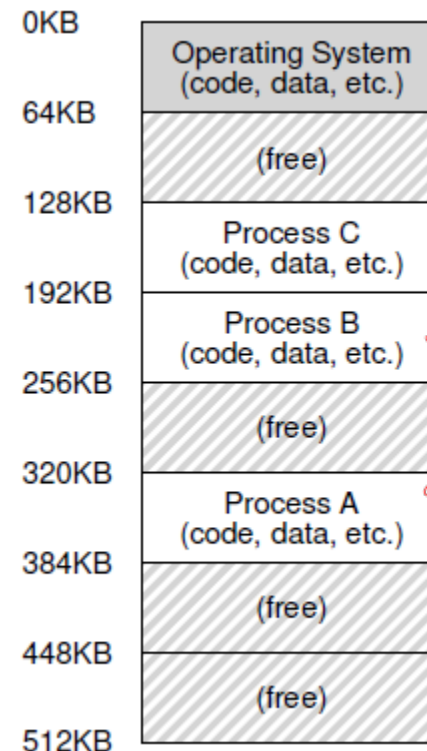
Chap 13, 14, 15, 16, 17

Address Spaces

- Easy to use abstraction of physical memory
- Running program's view of memory
- OS build this abstraction of a private, potentially large address space for multiple running processes (all sharing memory) on top of a single, physical memory



Virtual view

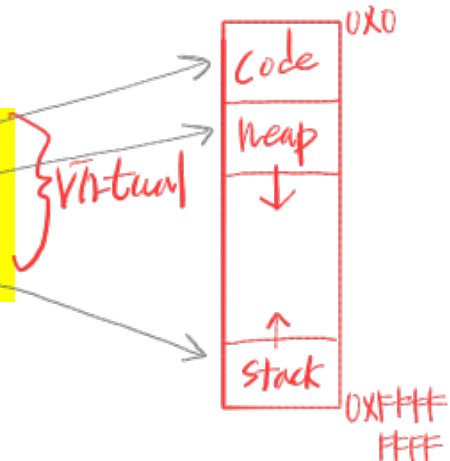


Physical memory in multiprogramming

Every Address You See is Virtual

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4     printf("location of code : %p\n", (void *) main);
5     printf("location of heap : %p\n", (void *) malloc(1));
6     int x = 3;
7     printf("location of stack : %p\n", (void *) &x);
8     return x;
9 }
```

location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack : 0x7fff691aea64



Types of Memory

- **Stack memory**

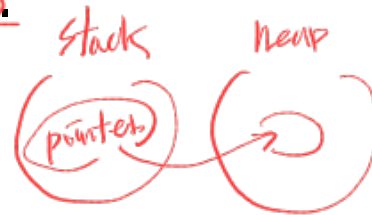
- Allocated/deallocated implicitly by the compiler.
- Called automatic memory.
- The compiler makes space on the stack when you call into ^{allocate} func().
- Returning from the function, the compiler deallocates the memory. (Local Variable)

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

- **Heap memory**

- All allocations and deallocations are explicitly handled by the programmer (malloc/free)
- The address itself is stored on the stack.

```
void func() {  
    int *x = (int *) malloc(sizeof(int));  
    ...  
}
```



Common Errors

- **Forgetting To Allocate Memory**

```
char *src = "hello";  
char *dst;           // oops! unallocated  
strcpy(dst, src);    // segfault and die
```

segmentation fault



```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src) + 1);  
strcpy(dst, src);    // work properly
```

YOU DID SOMETHING
WRONG WITH
MEMORY YOU FOOLISH
PROGRAMMER AND I AM
ANGRY

to make room for the
end-of-string character

- **Not Allocating Enough Memory (Buffer Overflow)**

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src)); // too small!  
strcpy(dst, src);    // work properly
```

*→ ok! fixed
buffer overflow
[memory] []
security issue
data corrupt*

- realloc() makes a new larger region of memory, copies the old region into it, and returns the pointer to the new region

- **Forgetting to Initialize Allocated Memory**

- Program will read from the heap some data of unknown value.
- calloc() allocates memory and also zeroes it before returning

003 초기화

Common Errors

- **Forgetting To Free Memory (Memory Leak)**

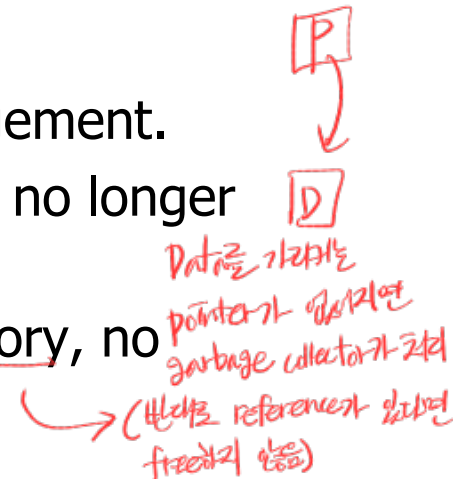
- Some languages support automatic memory management.

- Garbage collector figures out what memory you no longer have references to and frees it for you.

- If you still have a reference to some chunk of memory, no garbage collector will ever free it.

- Problem in long-running applications

- For a short-lived program, OS will clean up all of its allocated pages and thus no memory leak will take place.



- **Freeing Memory Before You Are Done With It (**Dangling Pointer**)**

- **Freeing Memory Repeatedly (**Double Free**)**

- **Calling free() Incorrectly**

- purify and valgrind will be helpful



Underlying OS Support

- **system calls used by memory library**

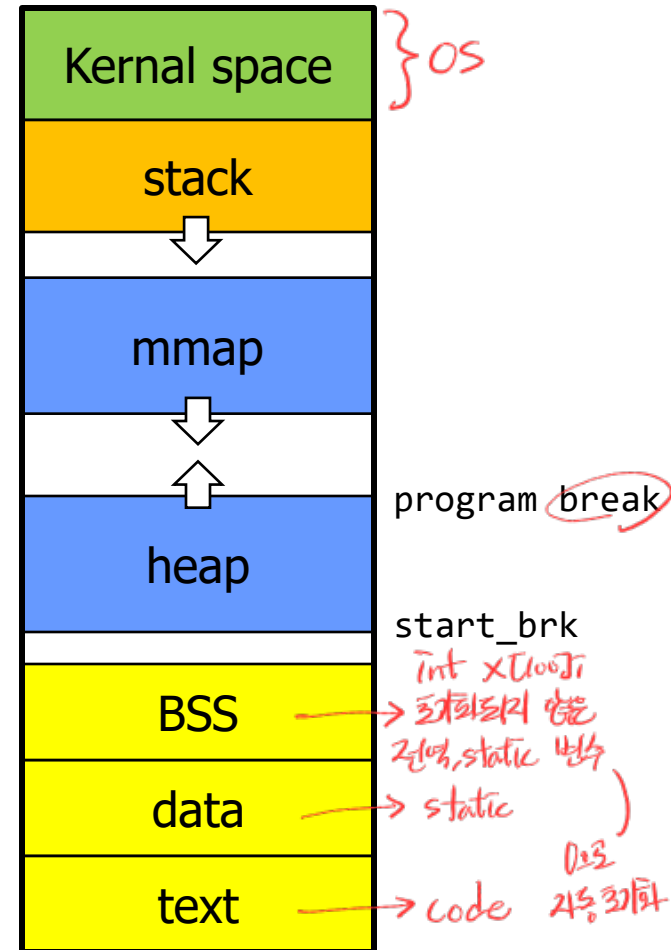
- **brk/sbrk**

- changes the location of the program's break: the location of the end of the heap - increases or decreases the size of the heap

heap의 끝
(heap의 크기)

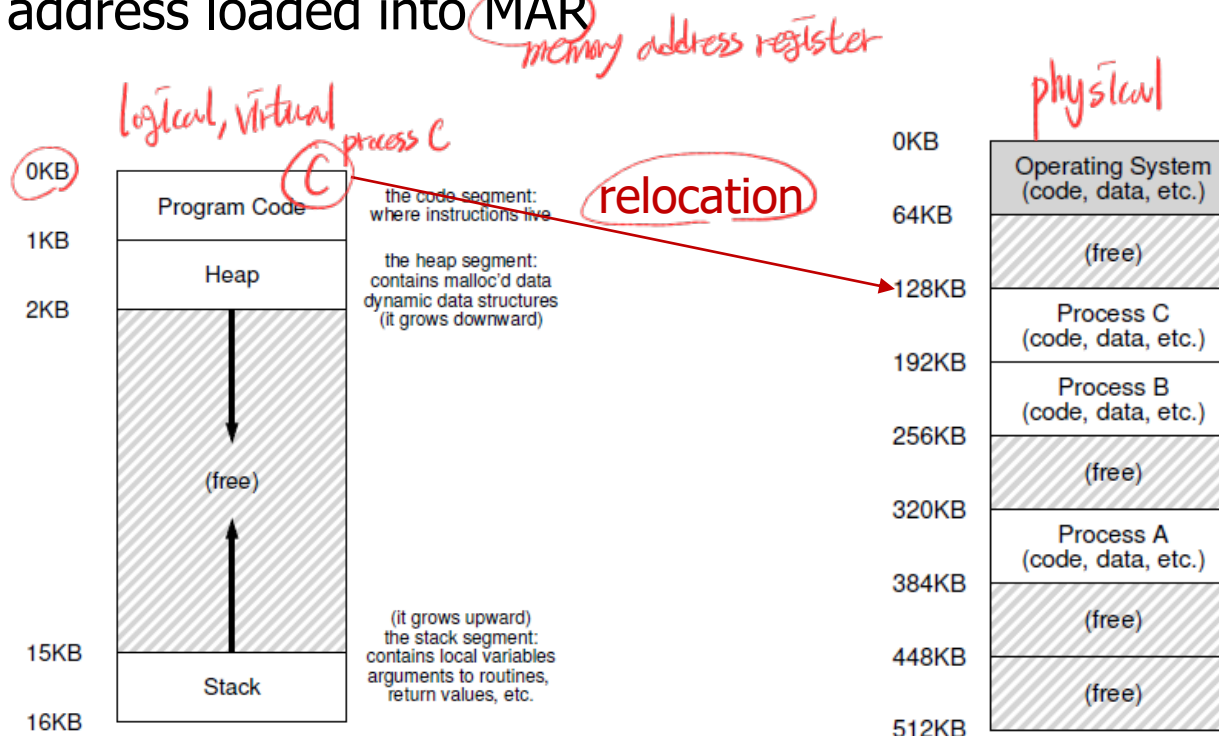
- **mmap**

- create an anonymous memory region within your program
 - a region which is not associated with any particular file but rather with swap space



Address Translation

- **Logical address (virtual address)**
 - An address generated by the CPU
- **Physical address**
 - An address seen by the memory unit
 - An address loaded into **MAR**



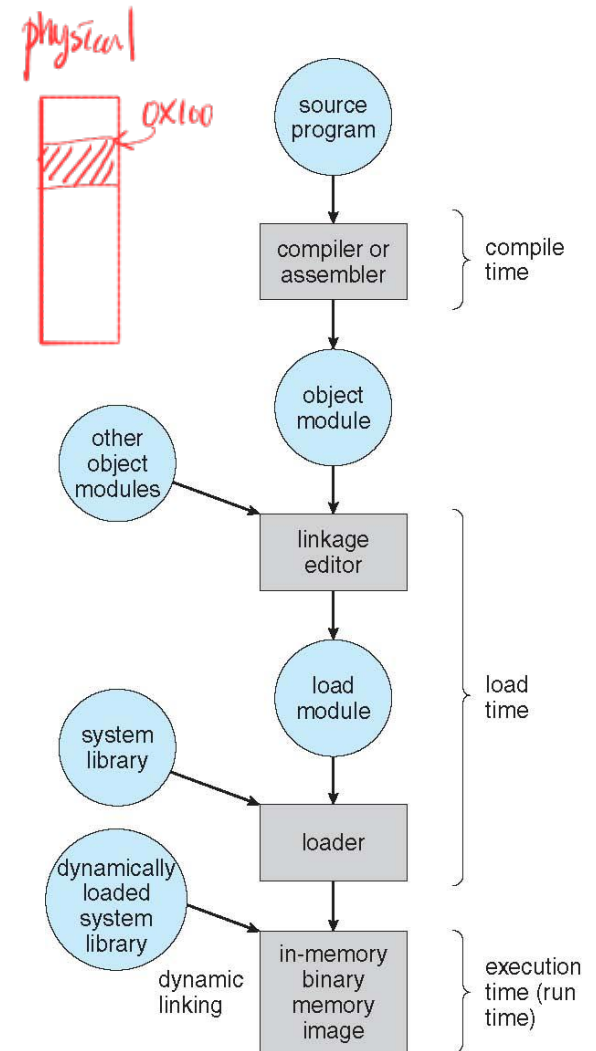
Static (Software-based) Relocation

- **Compile time binding**

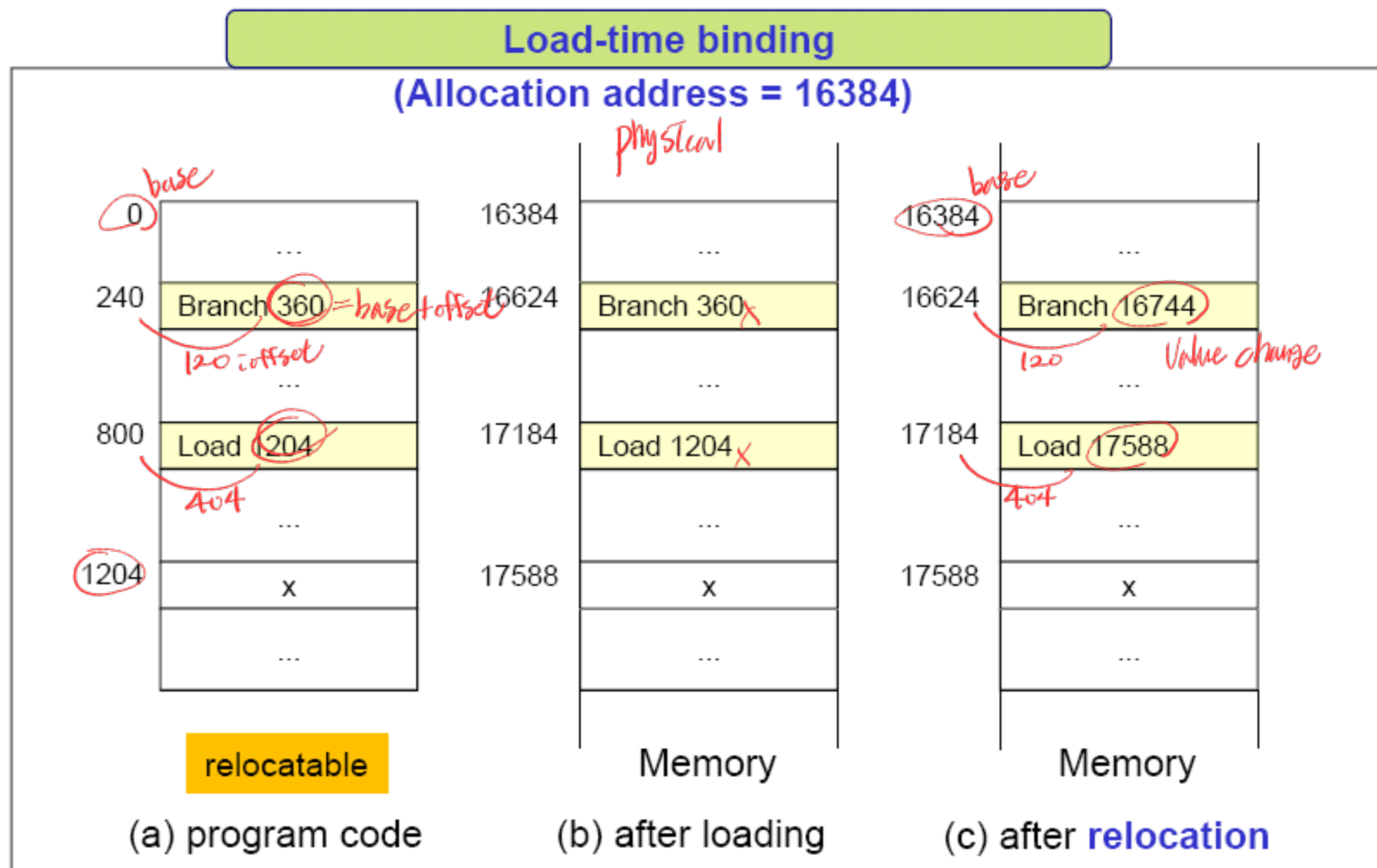
- When it is known at compile time where the process will reside in memory, then absolute code can be generated
- Changing the starting location requires recompilation
- MS-DOS .COM-format programs

- **Load time binding**

- When it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code
- Final binding is delayed until load time
- Changing the starting location requires reloading or relocation of the user code
- No protection
- difficult to later relocate an address space to another location



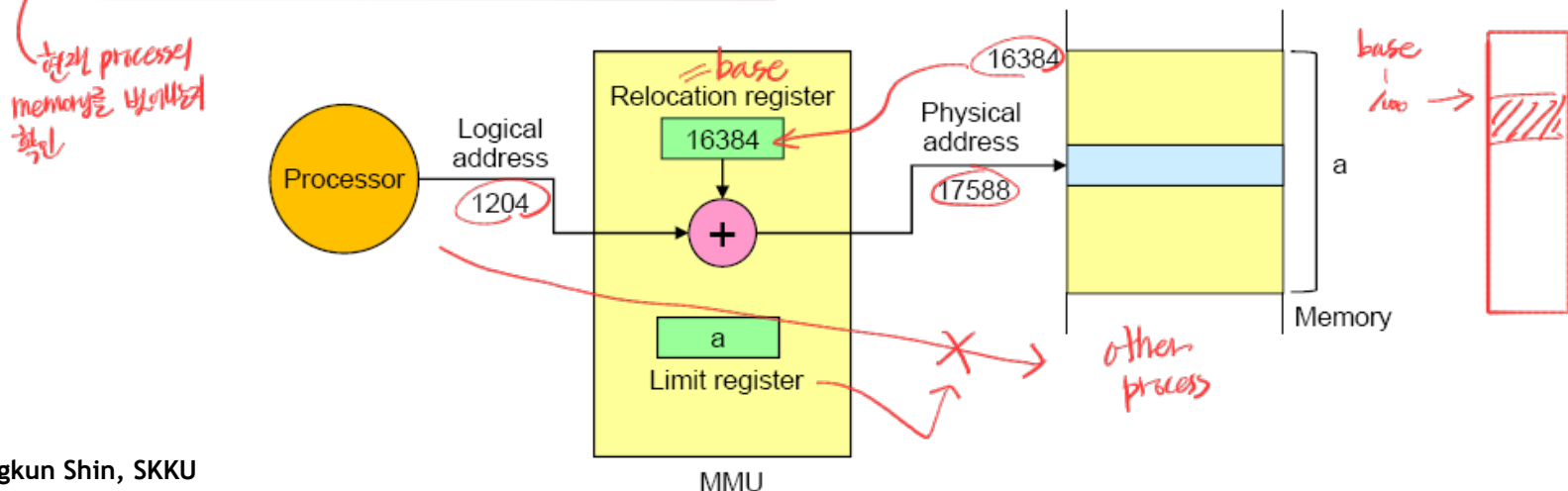
Address Binding



Dynamic (Hardware-based) Relocation

- **Runtime Binding**

- two hardware registers, base and bounds (limit), in MMU
- each program is written and compiled as if it is loaded at address zero.
- When a program starts running, the OS decides where in physical memory it should be loaded and sets the base register to that value.
- When any memory reference is generated by the process, it is **translated** by the processor in the following manner:
 - physical address = virtual address + base
- A **bounds** (or **limit**) register ensures that such addresses are within the confines of the address space.



Hardware Support

(kernel mode, supervise mode) ← system call User

- The hardware should provide special **privileged** instructions to modify the base and bounds registers, allowing the OS to change them when different processes run. A B context switching
- CPU must be able to generate **exceptions** in situations where a user program tries to access memory illegally (with an address that is "out of bounds")

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support <u>address translation</u> and <u>bounds checks</u></i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i><u>OS</u> must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i><u>OS</u> must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or <u>out-of-bounds memory</u></i>

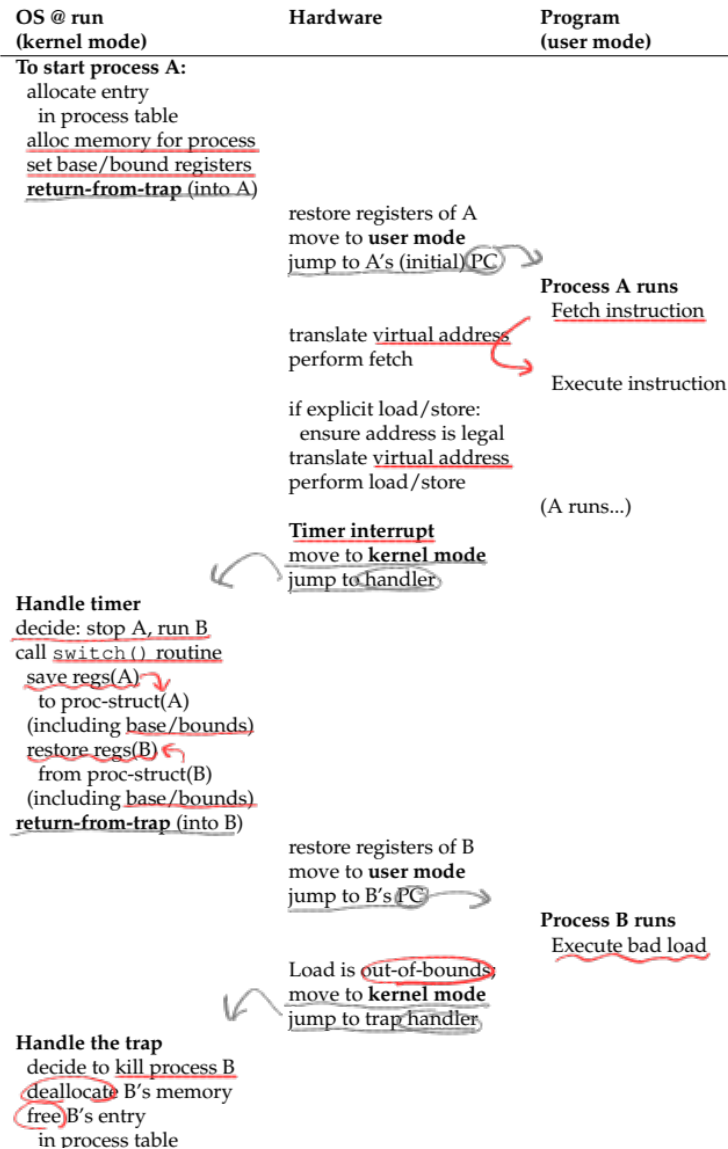
dynamic allocation of space

Operating System Issues

- OS must save and restore the base-and-bounds pair when it switches between processes

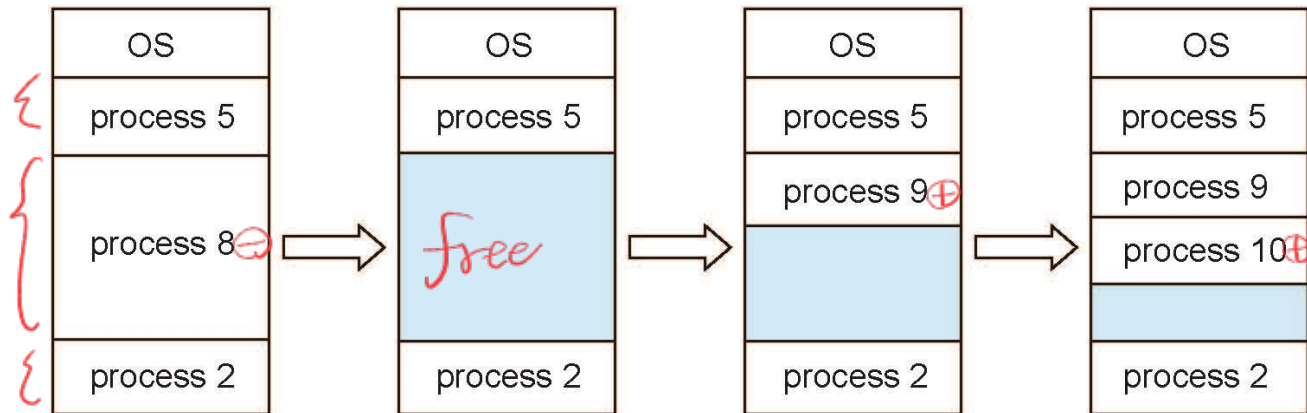
OS Requirements	Notes
Memory management	<i>Need to <u>allocate</u> memory for new processes; <u>Reclaim</u> memory from terminated processes; Generally manage memory via <u>free list</u></i>
Base/bounds management	<i>Must set base/bounds properly upon <u>context switch</u></i>
Exception handling	<i>Code to run when exceptions arise; likely action is to terminate offending process</i>

Exception Handling



Contiguous Memory Allocation

- Initially, all memory is available as a single large block of available memory (hole, partition)
- Contiguous allocation: when a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Memory partition state dynamically changes as a process enters (or exits) the system



Contiguous Memory Allocation

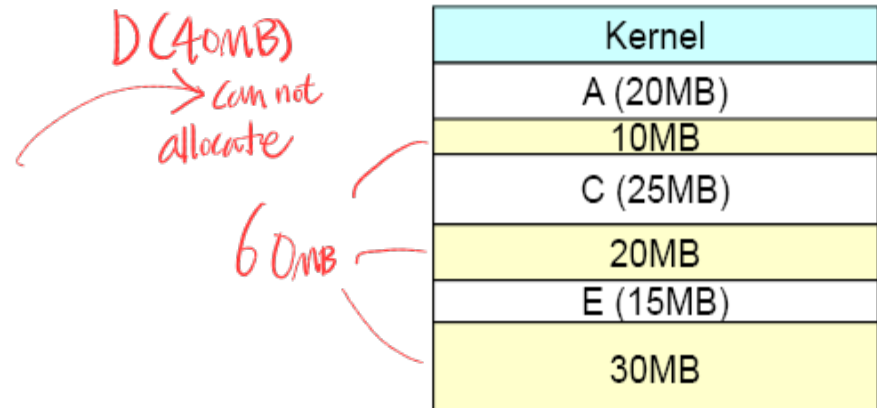
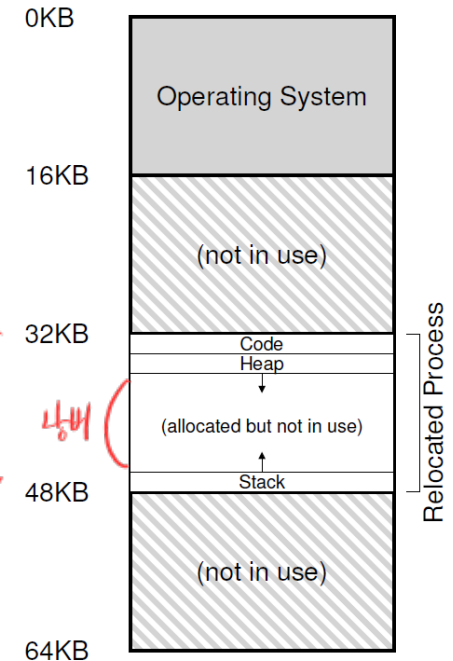
- **Fragmentation** *problem*

- **Internal** fragmentation

- When the process stack and heap are not too big, all of the space between the two is simply wasted

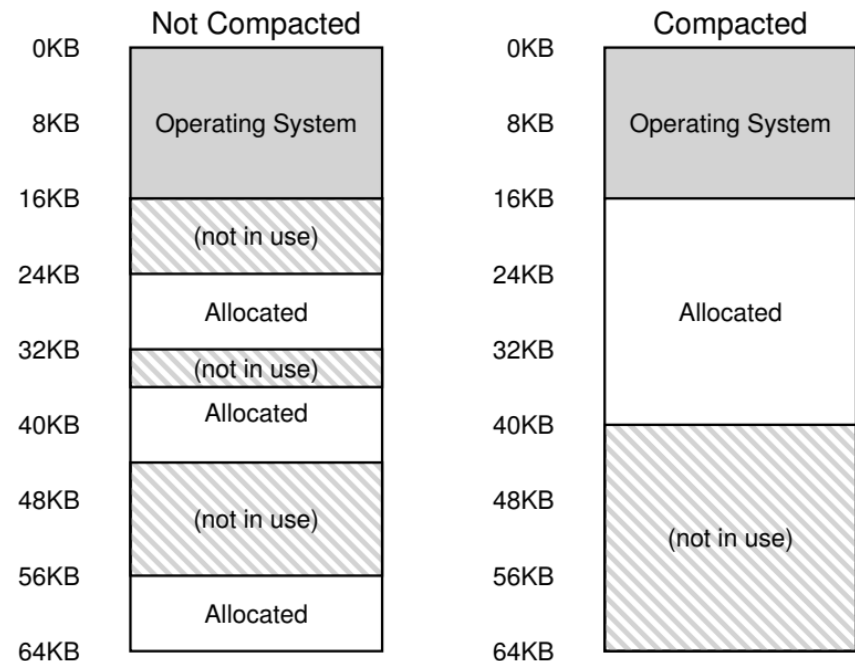
- **External** fragmentation

- Exists when enough total memory space exists to satisfy the request, but it is not contiguous



Contiguous Memory Allocation

- **Coalescing holes** *External Fragmentation*
 - Merge adjacent free partitions into one large partition
- **Compaction**
 - Shuffle the memory contents to place all free memory together in one large block (partition)
 - Done at execution time
 - Can be done only if relocation is dynamically possible
 - Consumes so much system resources
 - Consumes long CPU time



Contiguous Memory Allocation

- **Placement strategies**

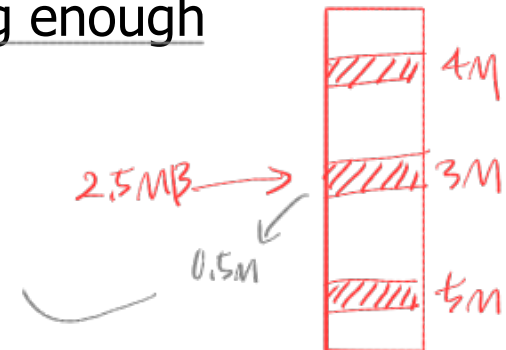
- First-fit

- Start searching at the beginning of the state table
- Allocate the first partition that is big enough
- Simple and low overhead

- Best-fit

- Search the entire state table
- Allocate the smallest partition that is big enough
- Long search time
- Can reserve large size partitions
- May produce many small size partitions
- External fragmentation

External fragmentation을 유발할 수 있음
→ 여러 작은 partitions



Contiguous Memory Allocation

- **Placement strategies**

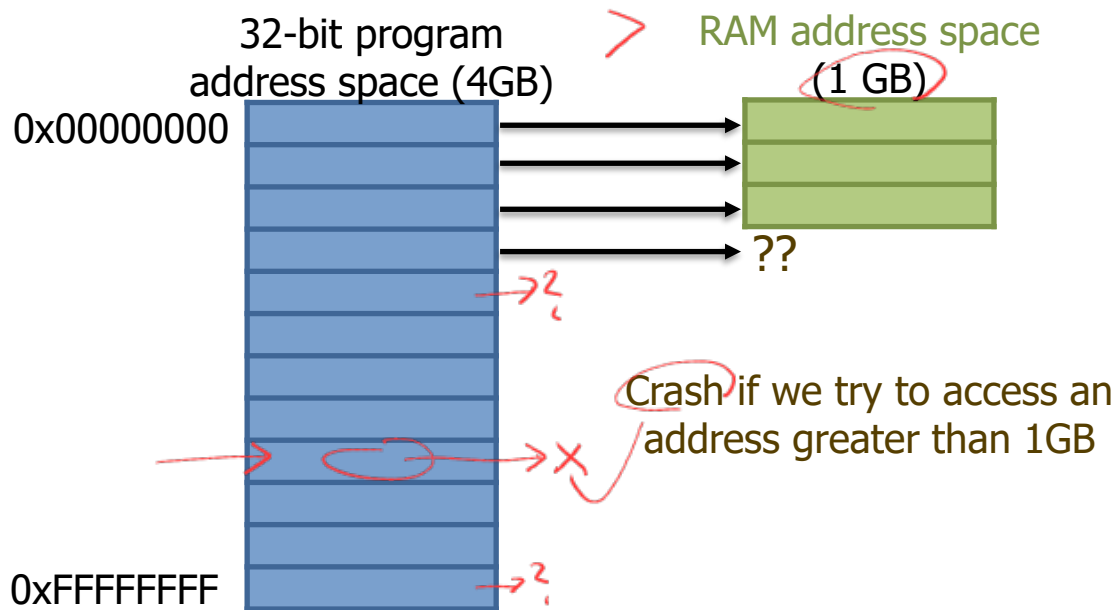
- Worst-fit *Best fit으로 인한 문제를 해결하기 위해 개발됨*
 - Search the entire state table
 - Allocate the largest partition available
 - May reduce the number of small size partitions
- Next-fit
 - Similar to first-fit method
 - Start searching from where the previous search ended
 - Circular search the state table
 - Uniform use for the memory partitions
 - Low overhead

Three Problems with Memory

Virtual memory가 없을 때 생기는 문제점 3가지,
" 가 해결하는 "

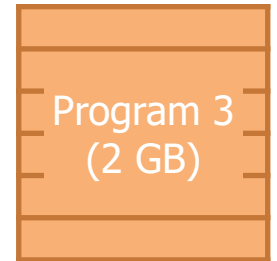
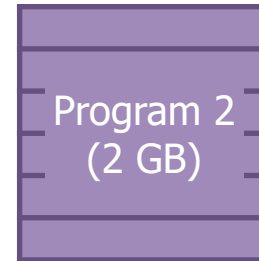
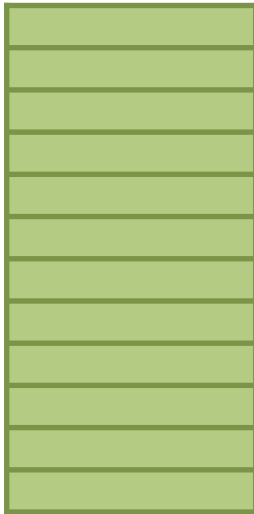
#1: Not Enough Space

- RISC-V32 provides a 32-bit address space
 - Q: How much memory can I access with a 32-bit address?
 - 2^{32} bytes = 4GB



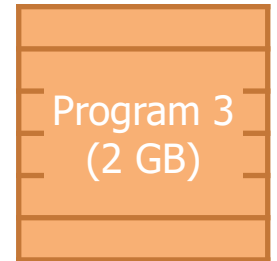
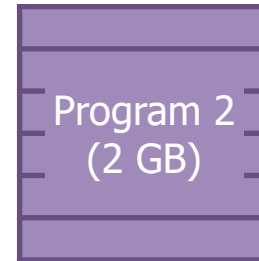
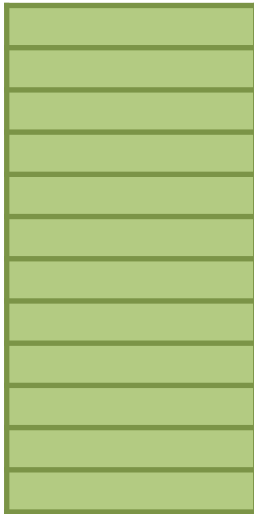
#2: Holes in Address Space

RAM address space
(4 GB)



#2: Holes in Address Space

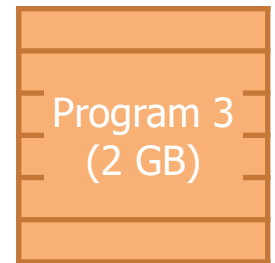
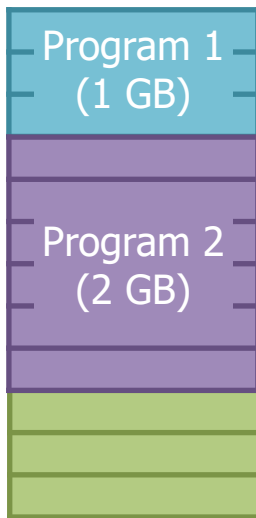
RAM address space
(4 GB)



1. Run Programs 1 and 2

#2: Holes in Address Space

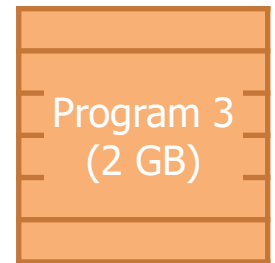
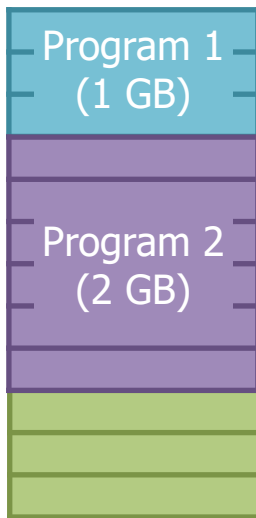
RAM address space
(4 GB)



1. Run Programs 1 and 2
(they use 3 GB of memory, leaving 1 GB free)

#2: Holes in Address Space

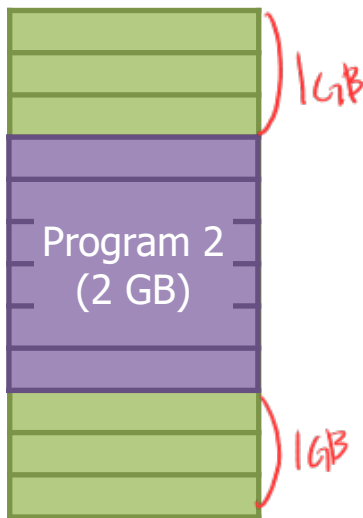
RAM address space
(4 GB)



1. Run Programs 1 and 2
(they use 3 GB of memory, leaving 1 GB free)
2. Quit Program 1

#2: Holes in Address Space

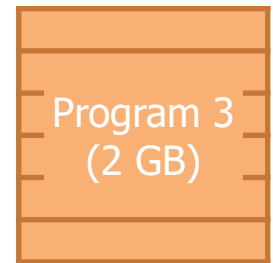
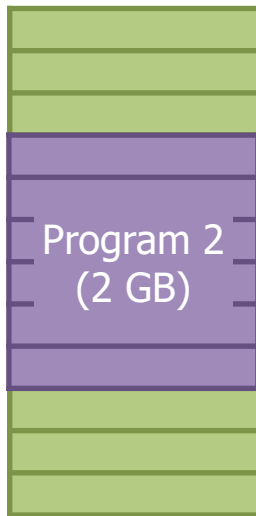
RAM address space
(4 GB)



1. Run Programs 1 and 2
(they use 3 GB of memory, leaving 1 GB free)
2. Quit Program 1
There are now 2GB free

#2: Holes in Address Space

RAM address space
(4 GB)



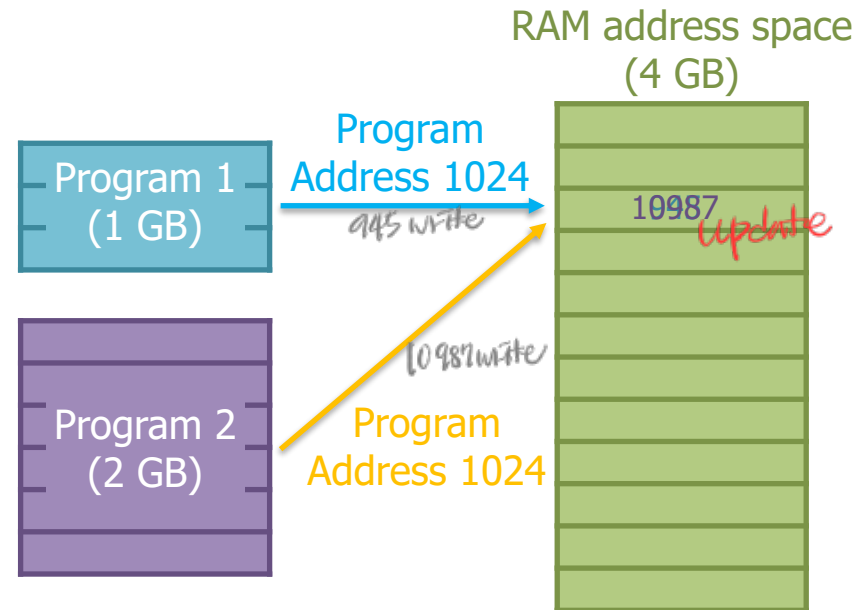
1. Run Programs 1 and 2
(they use 3 GB of memory, leaving 1 GB free)
2. Quit Program 1
There are now 2GB free
3. Try to run Program 3
- 3 We can't, even though there is enough space!

Memory
Fragmentation

#3: Ensuring Protection from Other Programs

- Each program can access any 32-bit memory address
- What if multiple programs access the same address?
- They can corrupt or crash each other

1. Program 1 stores your bank account balance at address 1024
2. Program 2 stores your video game score at address 1024



Problems with Memory

- If all programs have access to the same 32-bit address space
 - 1, Can crash if there is less than 4GB of RAM
 - 2, Can run out of space if we run multiple programs (Hole)
 - 3, Can corrupt other programs' data
- How do we solve this?
 - Give each program its own virtual address space

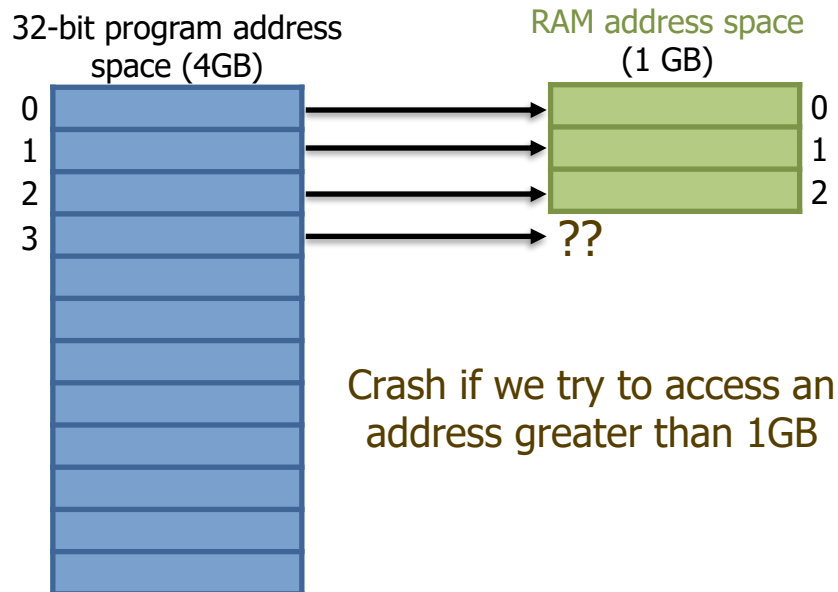
↓
physical

Virtual Memory: Indirection

Virtual memory takes ^{Virtual}program addresses and maps them to RAM addresses

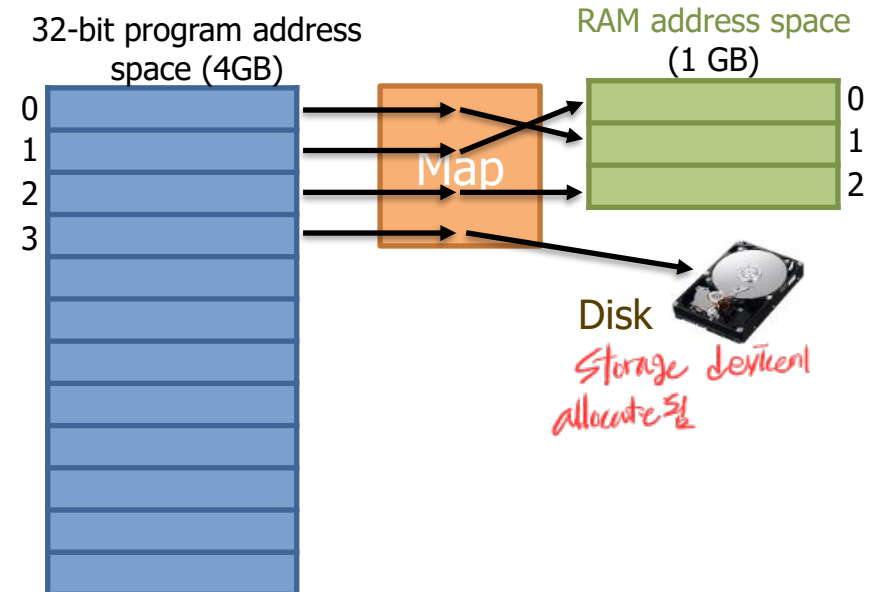
No Virtual Memory

program address = RAM address



Virtual Memory

program address maps to RAM address

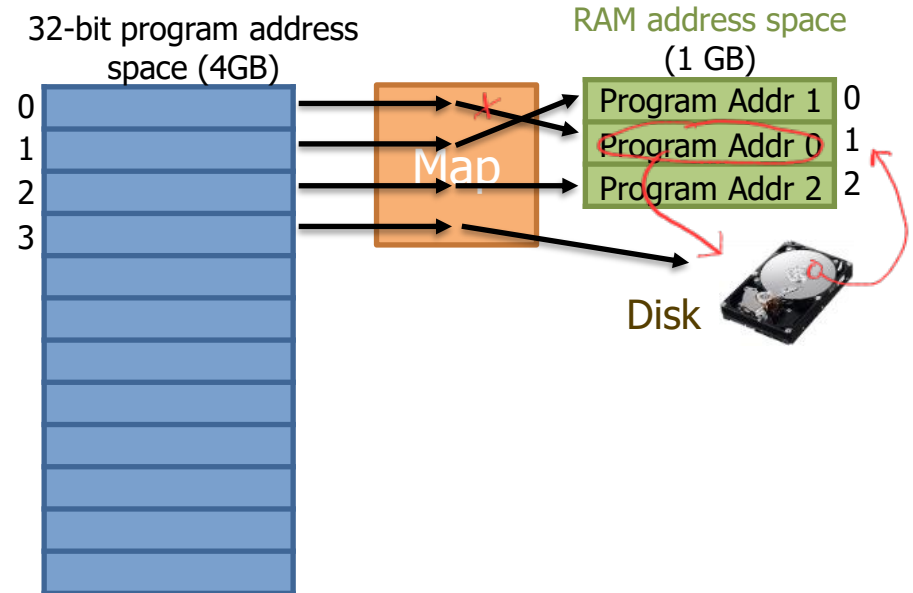


Solving Problem #1: Not Enough Memory

- Map some of the program's address space to disk
- When we need it, bring it into memory

1. Program accesses address 0
2. Program accesses address 1
3. Program accesses address 2
4. Program accesses address 3 *disk에서 가져옴*

Move out least recently accessed data
Bring in address 3 from disk

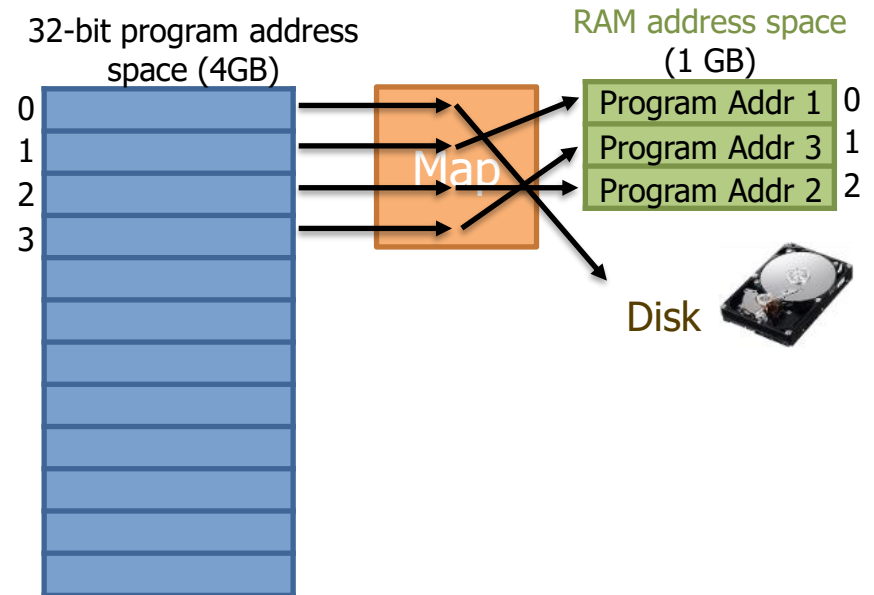


Solving Problem #1: Not Enough Memory

- Map some of the program's address space to disk
- When we need it, bring it into memory

1. Program accesses address 0
2. Program accesses address 1
3. Program accesses address 2
4. Program accesses address 3

Move out least recently accessed data
Bring in address 3 from disk



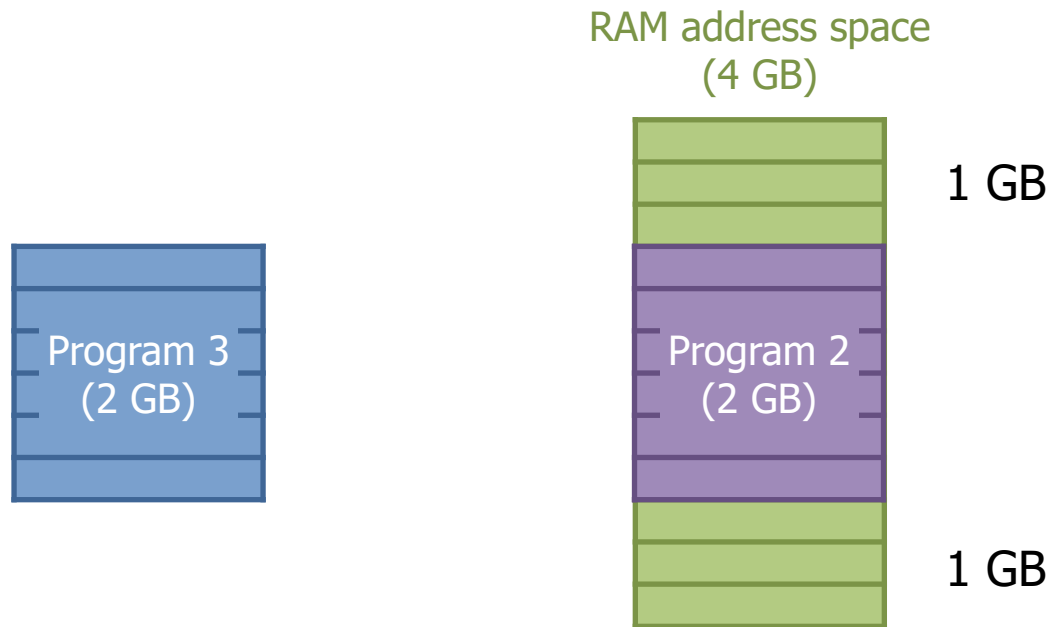
VM and Performance

- Q: What happens to the performance of the program when the data it needs is in the disk and not in main memory?
 - Performance decreases
 - Reading from disk is much slower than reading from RAM
 - Any time you can't fit your data in memory and have to go to disk you pay a HUGE performance penalty!
 - This is why buying more RAM makes your computer faster

main memory가 아닌 disk 이 읽으려고 성능↓
Ram이 클수록 computer가 빨라짐

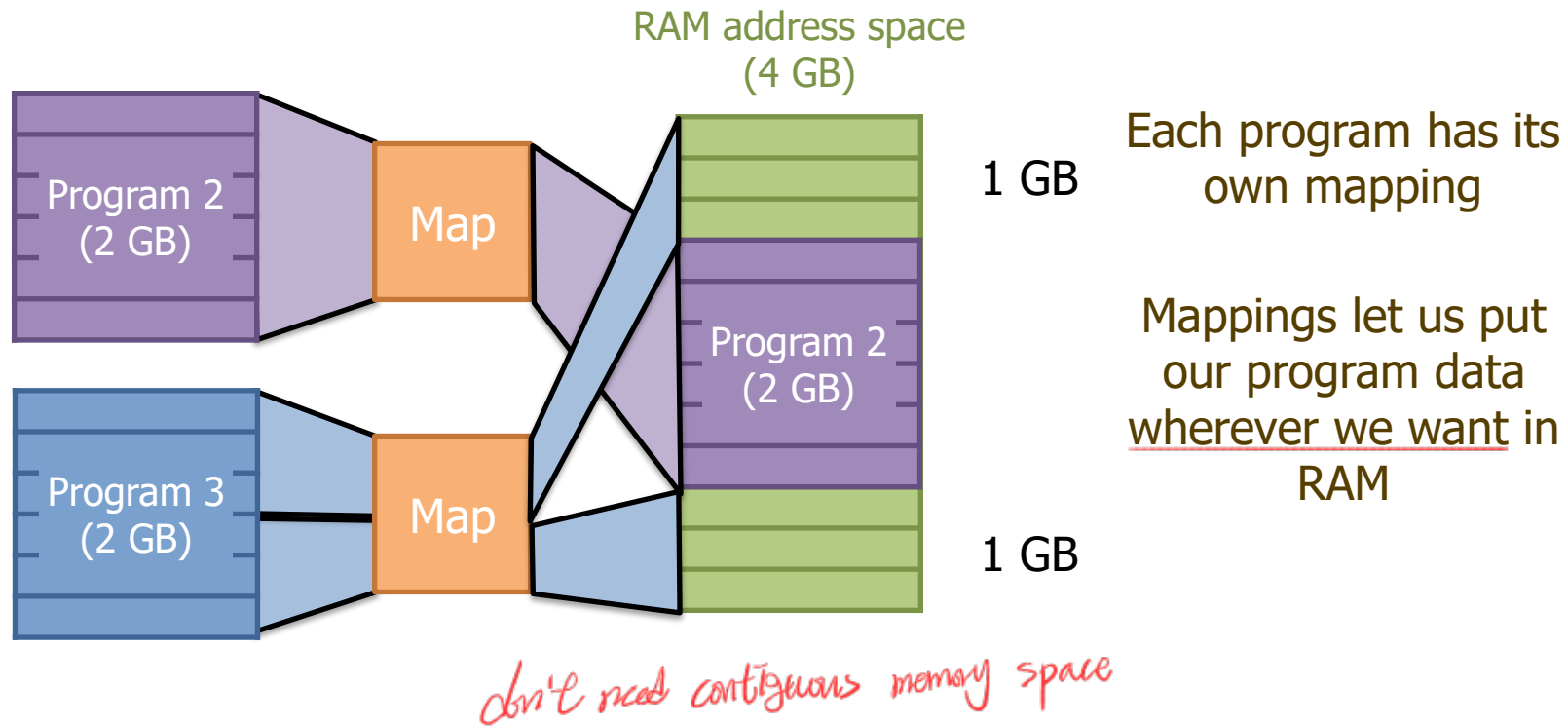
Solving Problem #2: Holes in Address Space

- How can we fit our program in the available space?



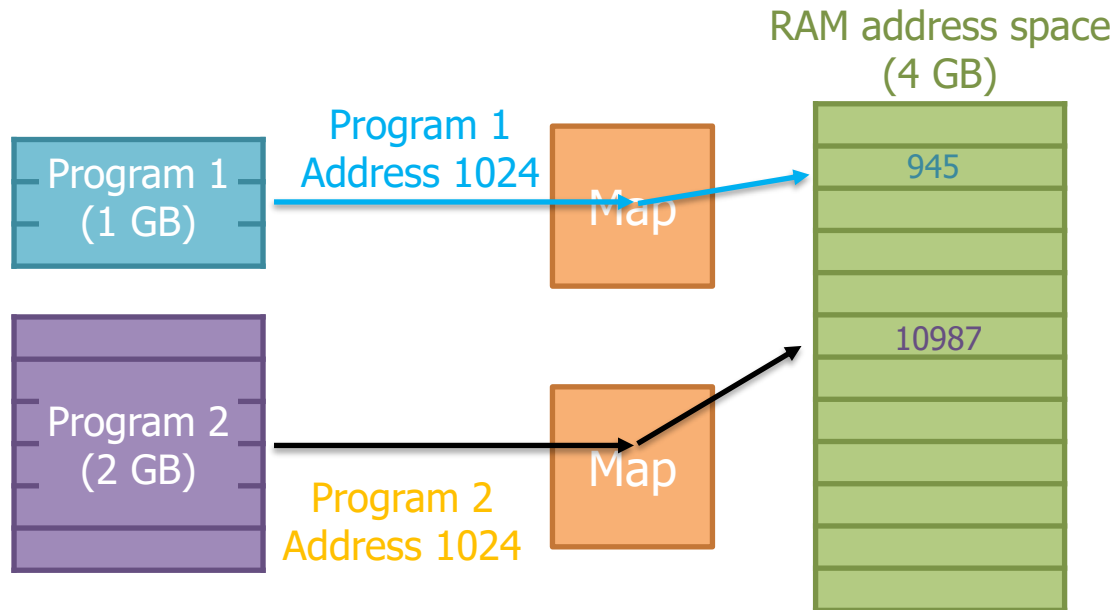
Solving Problem #2: Holes in Address Space

- How can we fit our program in the available space?



Solving Problem #3: Keeping Programs Secure

- Program 1's and Program 2's addresses map to different RAM addresses
- Because each program has its own address space, they cannot access each other's data



1. **Program 1** stores your bank balance at address 1024

VM maps it to **RAM address**

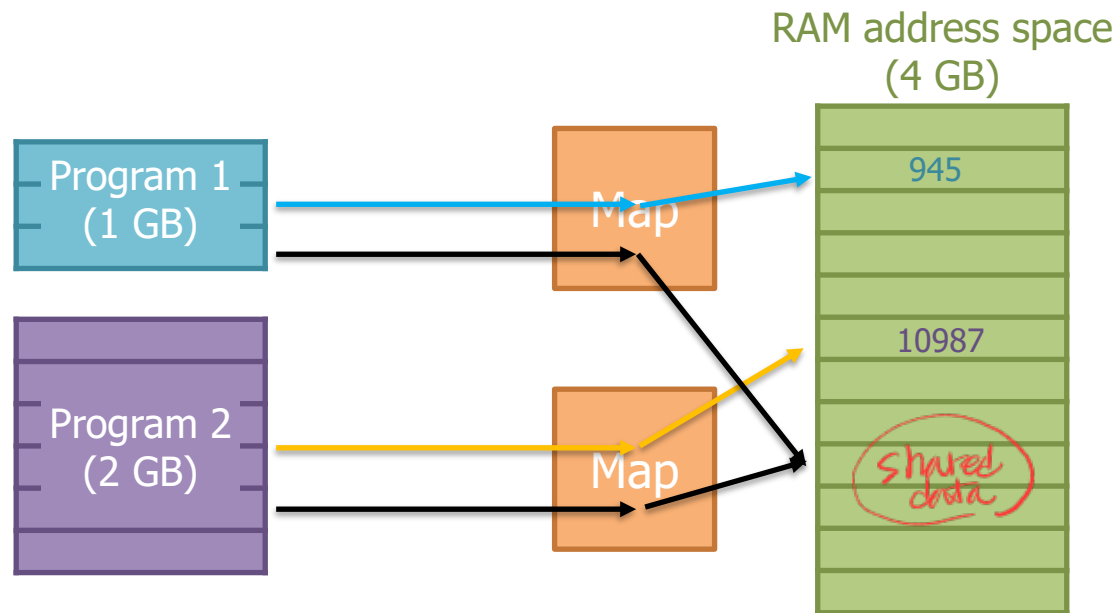
2. **Program 2** stores your video game score at address 1024

VM maps it to **RAM address**

Neither can touch the other's data!

Sharing Data Between Programs

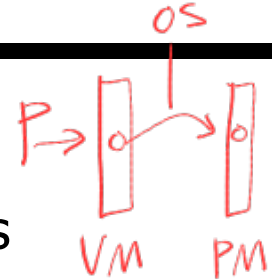
- Programs share a lot of data (eg libraries)
- VM enables us to easily share data while protecting private data!



Goals of Memory Virtualization

- **Transparency**

*program only access its virtual memory space,
It does not need consider the physical reality*



- Program shouldn't be aware of the fact that memory is virtualized
- Program behaves as if it has its own private physical memory
- OS (and hardware) does all the work to multiplex memory among many different jobs, and hence implements the illusion

- **Efficiency**

- Time: not making programs run much more slowly

- Need H/W support (TLB)

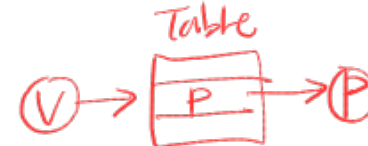
translation을 빨리 처리해 주어야 할

Translation Lookahead Buffer (cache translation entry)

- Space: not using too much memory for structures needed to support virtualization

optimize

translation table



- **Protection**

- protect processes from one another as well as the OS itself from processes
- deliver the property of isolation among processes



Discontiguous Memory Allocation

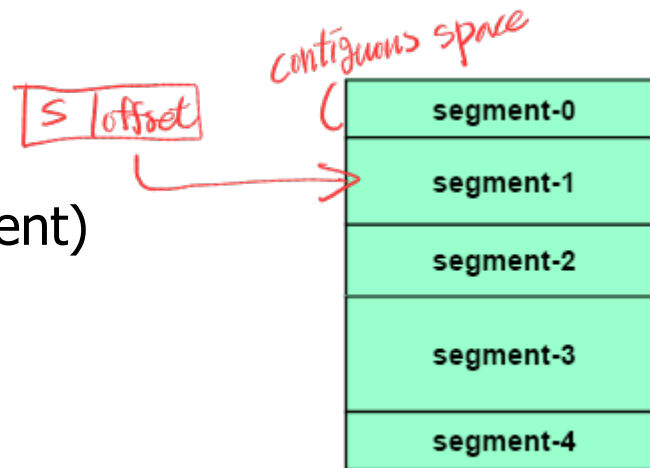
2 types of Discontiguous memory allocation In virtual memory technique

- **Segmentation**
- **Paging**

Segmentation

- **Basic concept**

- View memory as a collection of variable-sized segments
- View program as a collection of various objects
 - Functions, methods, procedures, objects, arrays, stacks, variables, and so on
- Logical address
 - $v = (s, d)$
 - s: segment number
 - d: offset (displacement)



Segmentation

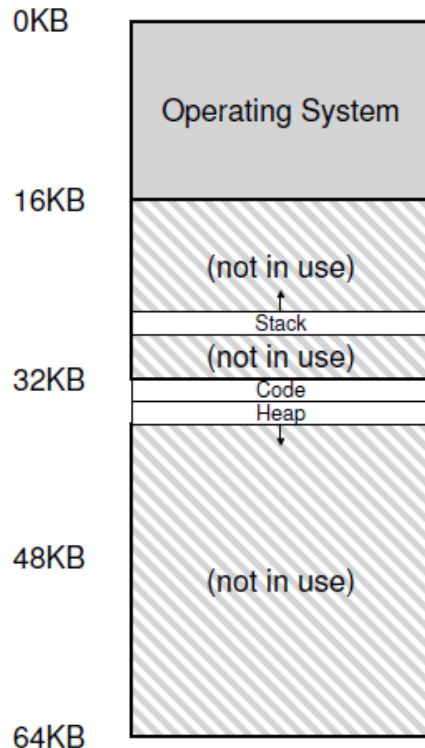
- **Basic concept**

- Normally, when the user program is compiled, the compiler automatically constructs segments reflecting the input program

- Code
 - Global variables
 - Heap
 - Stacks
 - Standard library

Segmentation

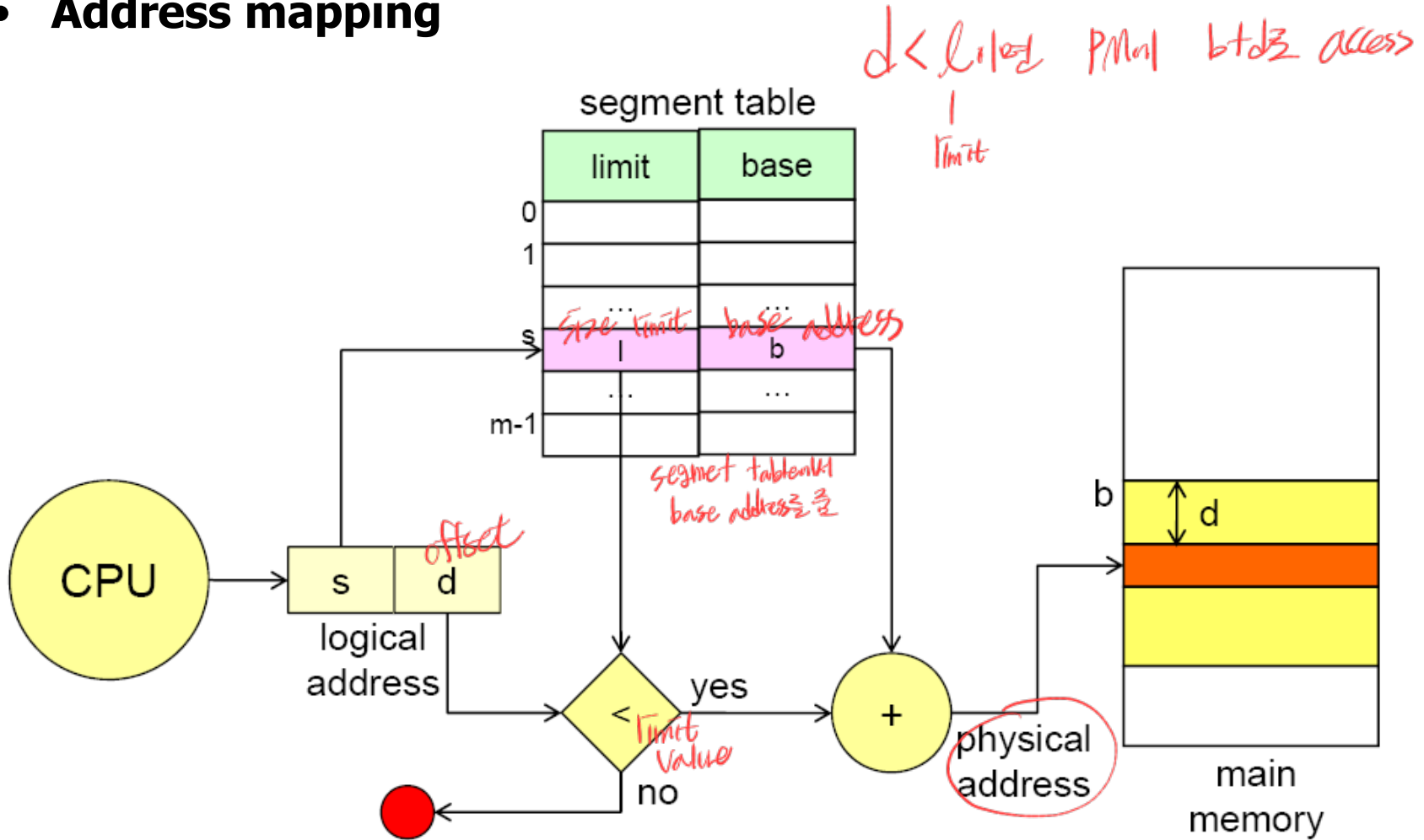
- All the unused space between the stack and the heap need not be allocated in physical memory,
- allowing us to fit more address spaces into physical memory.



<u>Segment</u>	Base	Size
<u>Code</u>	<u>32K</u>	<u>2K</u>
Heap	34K	2K
Stack	28K	2K

Segmentation

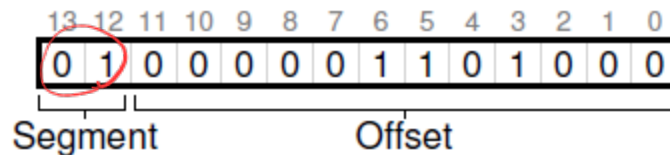
- Address mapping



segmentation violation or segmentation fault

Which Segment Are We Referring To?

- **Explicit** approach
 - the address space into segments is determined based on the top few bits of the virtual address



- **Implicit** approach
 - hardware determines the segment by noticing how the address was formed.
 - If the address was generated from the program counter (i.e., it was an instruction fetch), then the address is within the code segment
 - If the address is based off of the stack or base pointer, it must be in the stack segment
 - any other address must be in the heap.

What About The Stack?

- Stack grows backwards

Segment	Base	Size	Grows Positive?
Code	32K	2K	1 ↑
Heap	34K	2K	1 ↑
Stack	28K	2K	0 ↓ <i>낮은 주소</i>

Handwritten notes:
32~34KB (above Code and Heap)
34~36KB (below Heap and Stack)

Stack range: 26KB~28KB

Support for Sharing

- Hardware also has to check whether a particular access is permissible

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

Fine-grained vs. Coarse-grained Segmentation

- **Coarse-grained**
 - just a few segments (i.e., code, stack, heap)
- **Fine-grained**
 - requires segment tables to support the creation of a very large number of segments

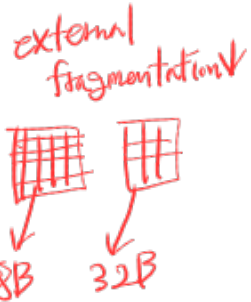
OS Support

- **What should the OS do on a context switch?**
 - the segment registers must be saved and restored.
 - each process has its own virtual address space, and the OS must make sure to set up these registers correctly before letting the process run again.
- **Segment Resizing**
 - For sbrk() syscall, updating the segment size register to the new (bigger) size.

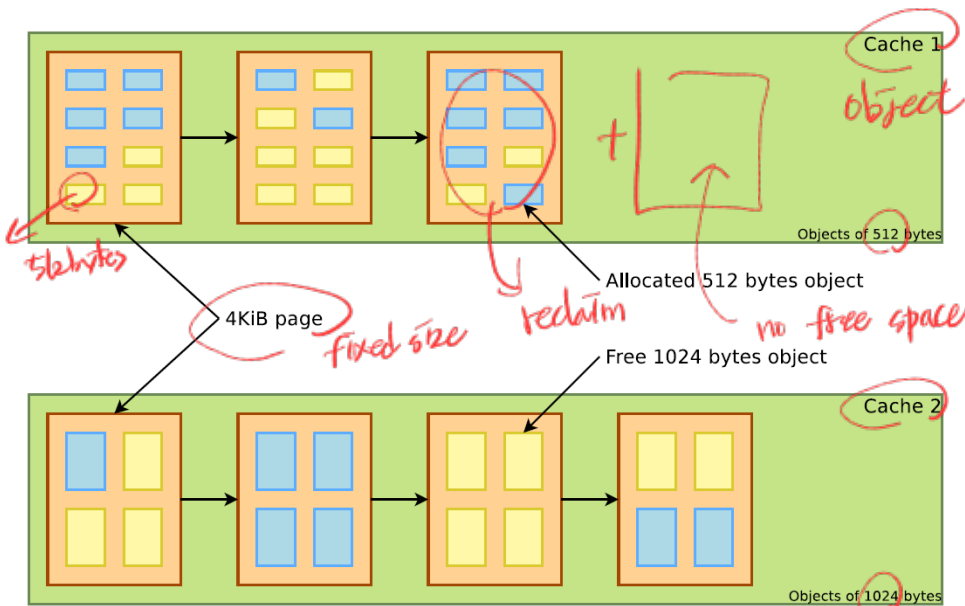


OS Support

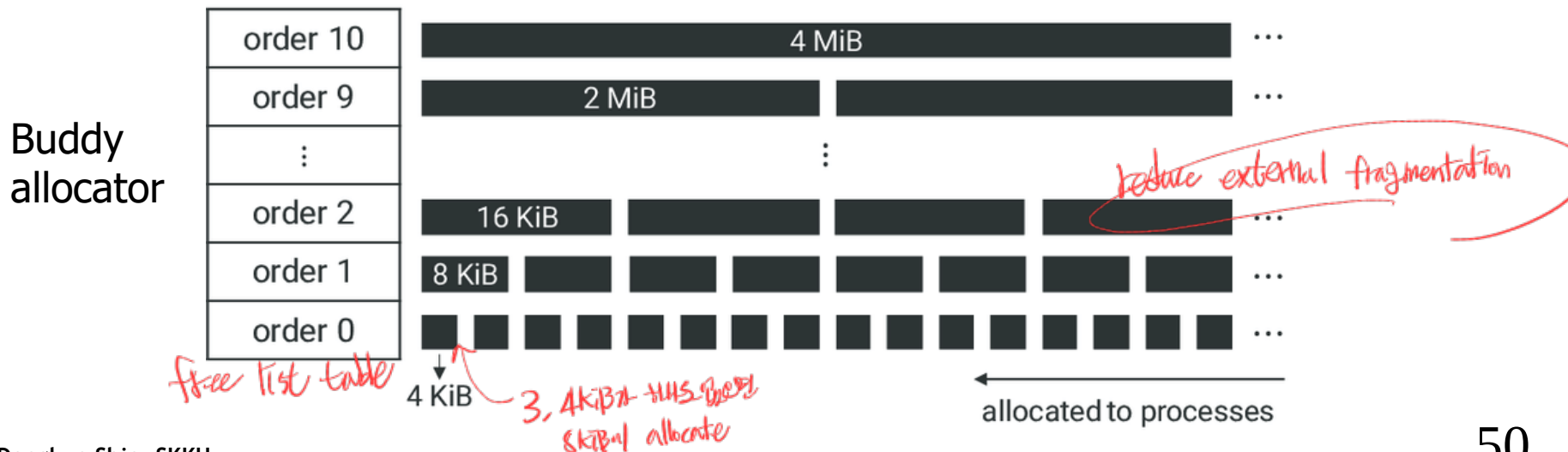
- Managing free space in physical memory
 - OS must find space in physical memory for its segments
 - best-fit, worst-fit, first-fit, ... → External fragmentation, need compaction — high overhead
 - segregated list
 - manage objects of popular size; all other requests are forwarded to a more general memory allocator
 - No fragmentation, no complicated search
 - how much memory should one dedicate to the pool of memory that serves specialized requests of a given size ?
 - slab allocator: allocates a number of **object caches** for kernel objects that are likely to be requested frequently (such as locks, file-system inodes, etc.)
 - when a given cache is running low on free space, it requests some **slabs** of memory from a more general memory allocator
 - when the reference counts of the objects within a given slab all go to zero, the general allocator can reclaim them
 - Binary buddy allocator



Slab Allocator & Buddy Allocator



```
% cat /proc/slabinfo
slabinfo - version: 1.1
kmem_cache           60      78      100      2      2      1
blkdev_requests      5120    5120     96    128    128     1
mnt_cache            20      40     96      1      1     1
inode_cache          7005   14792    480   1598   1849     1
dentry_cache         5469    5880    128    183    196     1
filp                  726     760     96     19     19     1
buffer_head          67131   71240     96   1776   1781     1
vm_area_struct        1204    1652     64     23     28     1
...
size-8192              1     17   8192      1    17     2
size-4096              41     73  4096     41    73     1
...
```



Homework

- Homework in Chap 13 (Debugging Memory Allocation w/ *valgrind*)

Quiz

1. Virtual memory allows processes to use more memory than physically available. (O)
2. The sbrk() system call is used to change the heap size. (O)
3. Memory allocated by malloc() can be resized using the resize() system call. (X) *realloc*
4. All processes share the same address space in a modern multitasking OS. (X)
5. Memory Management Unit (MMU) is primarily responsible for address relocation during execution. (O)
6. Dynamic address relocation is performed at program ~~load~~ time. (X)
7. Without address relocation, all programs would need to be loaded at the same memory location. (O)
8. The primary purpose of segmentation in memory management to divide memory into equal-sized pages. (X) *→ 다른 sizes divide*
9. In segmentation, a logical address is composed of segment number and offset. (O)
10. If a segment's limit is 500 and a program tries to access offset 520, segmentation fault occurs. (O)
11. malloc() is a library function that may internally call brk() or mmap(). (O)
12. The slab allocator help minimize external fragmentation. (O)
13. The slab allocator is suitable for allocating memory of varying and unpredictable sizes. (X)
14. The buddy allocator can suffer from internal fragmentation. (O)
15. In the buddy system, two buddies can be merged back into a larger block only if both are ~~allocated~~. (X)
16. Valgrind is commonly used to detect memory leaks in C programs. (O)
17. All modern operating systems automatically clean up leaked memory during execution. (X)